📑 Product Requirements Document (PRD)

Project Name: Sentinel Monitoring Dashboard (v1.0)

Status: Draft / Pending Implementation

Lead Engineer: Ahmed Hussein

Stack: React.js (Class Components), CSS3, JavaScript (ES6+)

1. Project Overview

The **Sentinel Dashboard** is a real-time monitoring interface designed to track the health and uptime of backend microservices. The system must simulate "heartbeat" checks and provide visual feedback on service failures.

2. Target Features (Functional Requirements)

2.1 Service Monitoring

- **Service Representation:** Each service must be encapsulated in a `ServiceCard` component.
- **Real-time Heartbeat:** Components must update their `uptime` (in seconds) every **1000ms** after mounting.
- **Failure Simulation:** Every second, there is a **5% probability** that a service transitions from `Online` to `Offline`.
- **Manual Recovery:** Each card must feature a **"Restart Service"** button. Clicking this resets the status to `Online` and clears the error state.

2.2 Global Dashboard

- **Aggregate View:** A `Dashboard` component that initializes three specific services: `Auth-API`, `Payment-Gateway`, and `Inventory-DB`.
- **Service Isolation:** A failure in one `ServiceCard` must **not** trigger a re-render or state change in another sibling card (Performance Optimization).

2.3 Cross-Cutting Concerns (HOC)

- **Audit Logging:** A Higher-Order Component (`withLogger`) must wrap the `ServiceCard`. It logs every state change or re-render to the console with a timestamp.

3. Technical Specifications (Non-Functional Requirements)

3.1 Architecture & Patterns

Requirement	Implementation Detail
Component Type	Strictly **Class Components** (to practice Lifecycle & `this` binding).
State Management	Local Component State (`this.state`).
Logic Reuse	Higher-Order Component (HOC) pattern.

Optimization	Use of `React.PureComponent` to prevent unnecessary diffing.

3.2 Resource Management

🎬 **Memory Safety:** All `setInterval` instances must be explicitly cleared during the `componentWillUnmount` phase.

🎬 **State Integrity:** State updates for the counter must use the **functional update pattern** to avoid race conditions during rapid re-renders.

4. Success Metrics & Validation (The "Evidence Checklist")

As per your engineering standards, the project is considered "Done" only when:

1. **Baseline vs. After:** Compare re-render counts when using `Component` vs. `PureComponent`.
2. **Memory Leak Test:** Verify in Chrome DevTools (Performance tab) that intervals stop when cards are removed.
3. **Big-O Analysis:** Verify that the Dashboard render time is $O(N)$ where N is the number of services.

5. UI/UX Design Constraints

- **Visual States:** * `Online`: Green text or border.
    - `Offline`: Red text or border + "Service Down" warning.
- **Typography:** Use a monospace font (like `Courier New`) to give it a "Terminal/DevOps" feel.

Execution Roadmap

Phase 1: The "Scaffolding" (Environment Setup) ✅

Before writing business logic, ensure your "Development Operations" (DevOps) are ready.

1. **Project Initialization:** Create the React app (if you haven't) using `npx create-react-app sentinel-dashboard`.
2. **Directory Architecture:** Organize your folders for scalability:
    - `/src/components` (For `Dashboard.js` and `ServiceCard.js`)
    - `/src/hoc` (For `withLogger.js`)
    - `/src/styles` (For your CSS)

Phase 2: Defining the "Data Contract" (State & Props)

Since you are a Java developer, think of this as defining your **DTOs (Data Transfer Objects)**. You must decide exactly what data flows through the system.

Your Task: Write down (in a comment or a notepad) the exact keys for:

- **Props:** What does `Dashboard` send to `ServiceCard`? (e.g., `serviceName`, `id`).
- **State:** What changes over time inside `ServiceCard`? (e.g., `status`, `uptime`).

Phase 3: The "Walking Skeleton" (Static UI)

Build the UI **without logic** first. This confirms your CSS and Component hierarchy are correct.

1. Create the `ServiceCard` class.
2. Hard-code the JSX to show a "Mock" service (e.g., `Auth-API | Status: Online | Uptime: 0s`).
3. Render three of these in the `Dashboard`.
4. **Verification:** Does it look like a dashboard? If yes, move to logic.

Phase 4: Implementing the "Heartbeat" (Lifecycle Logic)

This is where your `componentDidMount` and `setState` knowledge comes in.

1. **The Timer:** Inside `componentDidMount`, set up the 1-second interval.
2. **The Logic:** Use `Math.random()` to handle that 5% failure chance.
3. **The Safety Net:** Immediately implement `componentWillUnmount` to clear the interval. **Never write a "start" without writing the "stop" first.**

Phase 5: Cross-Cutting Concerns (The HOC)

Once the dashboard is "beating" (updating every second), wrap it in your `withLogger`.

1. Verify that the console shows logs without breaking the UI.
2. Check for "Prop Drilling" issues (ensure your HOC passes props down using `{...this.props}`).

📁 Professional Project Structure (Updated for PRD)

```
sentinel-dashboard/
├── public/
├── src/
│   ├── components/
│   │   ├── Dashboard.js       (The Master Controller)
│   │   └── ServiceCard.js     (The Instrumented Component)
│   ├── hoc/
```

```
│   │   └── withLogger.js      (The Audit Logging Wrapper)
│   ├── styles/
│   │   ├── App.css            (Global Layout & Variables)
│   │   └── ServiceCard.css    (Component-Specific Styling)
│   ├── App.js                 (Entry Point for UI)
│   ├── index.js               (React DOM Bootstrapping)
│   └── setupTests.js          (Testing Configuration)
├── package.json
└── README.md                  (Project Documentation)
```

📝 Finalized Data Contract (Phase 2)

Based on the requirements in the PRD, we will define the component interface as follows:

1. Component Props (Static Data from Dashboard)

These are passed into the ServiceCard and do not change throughout its life.

- **serviceName (String):** The name of the service (e.g., "Auth-API").
- **id (String/Number):** A unique identifier for the service instance.

2. Component State (Dynamic Data inside ServiceCard)

These values will change over time and trigger UI re-renders.

- **status (String):** Initialized to "Online". We use a String to match the "Online" and "Offline" visual states required by the UI constraints.
- **uptime (Number):** Initialized to 0. This will increment every second to track the service's active duration.

🏛️ Data Contract Summary Table

Category	Key	Data Type	Initial Value
Props	serviceName	String	(Passed by Dashboard)
Props	id	String	(Passed by Dashboard)
State	status	String	"Online"
State	uptime	Number	0